Fixed Points, Feedback, and Adaptive Computation

Sergey Plis

Professor of Computer Science
Director of Machine Learning, TReNDS center
Georgia State University

Abstract

A standard deep network applies the same fixed sequence of operations to every input, paying the same cost whether the input is trivial or hard. This "ballistic" computation cannot loop back to refine an answer, check its own intermediate results, or hold state beyond a single forward pass. This course replaces that rigid design with iterative, stateful computation - through recurrence, equilibrium solving, adaptive depth, explicit memory, and learned test-time updates - so that models can spend effort where it is most needed.

Course Objective: By the end of this course, students will be able to recognize when a modeling problem calls for a standard feed-forward approximation and when it instead requires an explicit computational process involving state, iteration, or adaptation.

Contents

1 Fixed Points, Feedback, and Adaptive Computation **3**

1.1 Lecture 1 – Architectures, Recurrence, and Automatic Differentiation . . . 3

1.1.1 Core readings . . . 4

1.1.2 Optional readings . . . 4

1.1.3 Expected takeaways . . . 4

1.2 Lecture 2 – Fixed-Point Computation and Implicit Differentiation . . . 5

1.2.1 Core readings . . . 5

1.2.2 Optional readings . . . 5

1.2.3 Expected takeaways . . . 6

1.3 Lecture 3 – Adaptive Computation: Halting, Routing, and Dynamic Depth . . . 6

1.3.1 Core readings . . . 7

1.3.2 Optional readings . . . 7

1.3.3 Key takeaways . . . 7

1.4 Lecture 4 – Iterative Refinement and Introspective Computation 8

1.4.1 Core readings . . . 8

1.4.2 Optional readings . . . 9

1.4.3 Key takeaways . . . 9

1.5 Lecture 5 – Memory, State Tracking, and Long-Horizon Reasoning . . . 11

1.5.1 Core readings . . . 11

1.5.2 Optional readings . . . 11

1.5.3 Key takeaways . . . 12

1.6 Lecture 6 – Test-Time Learning, Self-Modification, and Nested Learners . . . 12

1.6.1 Core readings . . . 12

1.6.2 Optional readings . . . 13

1.6.3 Key takeaways . . . 13

1 Fixed Points, Feedback, and Adaptive Computation

A standard deep network applies a fixed sequence of transformations to every input — the same depth, the same cost, regardless of whether the input is trivial or ambiguous. This "ballistic" computation cannot iterate toward a solution, check its own intermediate results, or maintain state beyond a single forward pass. Yet these are precisely the capabilities needed for hard reasoning, long-horizon dependencies, and efficient inference.

The unifying theme of this course is replacing fixed-depth feed-forward computation with iterative, stateful computation — whether via recurrence, equilibrium solving, adaptive depth, explicit memory, or learned test-time updates — so that models can allocate effort where it is needed.

Throughout the course, we will compare these approaches not only by accuracy but by compute allocation, convergence behavior, memory footprint, and their ability to generalize as *processes* rather than fixed input–output maps. We will also pay sustained attention to where these methods break: unstable dynamics, solver dependence, truncation bias, poorly calibrated halting, memory blow-up, and train/test mismatch in compute budgets.

We start from classical architectures and the mathematical tools needed to differentiate through them (Lecture 1), show how to compute gradients at fixed points and equilibria (Lecture 2), then progressively build toward models that adaptively choose their own depth (Lecture 3), introspect on and refine their own internal states (Lecture 4), manage memory over long horizons (Lecture 5), and ultimately learn their own learning and update rules (Lecture 6).

By the end of the course, students should be able to recognize when a modeling problem calls for a fixed feed-forward approximation and when it instead calls for an explicit computational process with state, iteration, or adaptation.

1.1 Lecture 1 – Architectures, Recurrence, and Automatic Differentiation

We establish the baseline: standard deep networks as fixed-depth, feed-forward computation graphs. We review FFNNs, RNNs, and Transformers, emphasizing RNNs as dynamical systems — the first architecture to break the fixed-depth mold. We close with automatic differentiation as the universal tool for computing gradients through arbitrary computation, preparing for the iterative and fixed-point settings of later lectures.

1.1.1 Core readings

- Vaswani et al., "Attention Is All You Need"
- Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks" (also in pdf)
- Baydin et al., "Automatic Differentiation in Machine Learning: A Survey"

1.1.2 Optional readings

- Lipton, "A Critical Review of Recurrent Neural Networks for Sequence Learning"
- Goodfellow et al., "Sequence Modeling: Recurrent and Recursive Nets" (Deep Learning, Ch. 10)
- Olah, "Understanding LSTM Networks"
- Instructor Video Primers on Automatic Differentiation
  - AD intro
  - Forward Mode AD
  - Reverse Mode AD

1.1.3 Expected takeaways

- Recognize that standard feed-forward networks (including Transformers) perform a fixed amount of computation per input, regardless of difficulty — the "ballistic" paradigm that the rest of this course challenges.
- Understand the high-level structure of Transformer encoders/decoders. Recognize that the attention matrix is not a fixed learned parameter — it is *computed on the fly from the input*, making attention a data-dependent operation. This matters because later lectures will show that these on-the-fly computations can themselves be iteratively refined (L4/SELF-Transformer), used as memory access (L5), or adapted at test time (L6).
- See RNNs as **dynamical systems over sequences**: unrolling in time, backpropagation through time, hidden state as a lossy summary of

the past. Recognize that a convergent recurrent network can, when iterated to convergence, be viewed as computing a fixed point of its state-transition map - a central idea formalized in Lecture 2.

- Understand automatic differentiation (forward and reverse mode) as the general tool for computing gradients of arbitrary computation graphs, preparing for differentiating through iterative and fixed-point computations in later lectures.

1.2 Lecture 2 – Fixed-Point Computation and Implicit Differentiation

We build working familiarity with the mathematical tools — contraction mappings (Banach fixed-point theorem) and the implicit function theorem — then show how they underpin a 35-year arc of differentiating through convergent dynamics: from Almeida–Pineda (1987) through Recurrent Back-Propagation to modern Deep Equilibrium Models.

1.2.1 Core readings

- "Implicit Functions and Automatic Differentiation" (Implicit Layers Tutorial)
- Pineda, "Generalization of Backpropagation to Recurrent and Higher Order Networks"
- Liao et al., "Reviving and Improving Recurrent Back-Propagation" (greatly clarified the above)
- Bai et al., "Deep Equilibrium Models"

1.2.2 Optional readings

- "Deep Equilibrium Models" chapter (Implicit Layers Tutorial)
- Chen et al., "Neural Ordinary Differential Equations" — continuous-depth networks trained via the adjoint method; the key mathematical ideas (adjoint sensitivity analysis) predate this work and are already present in Pineda (1987) and the optimal control literature
- Chang et al., "Object Representations as Fixed Points"
- Wu et al., "Separation and Bias of Deep Equilibrium Models on Expressivity and Learning Dynamics"

- Lorraine et al., "Optimizing Millions of Hyperparameters by Implicit Differentiation"

1.2.3 Expected takeaways

- Know the Banach fixed-point theorem (contraction mappings guarantee existence, uniqueness, and iterative convergence to a fixed point) and the implicit function theorem (conditions under which we can differentiate through the solution of $F(z, \theta) = 0$) as the mathematical foundations for the rest of the course.

- Understand Almeida–Pineda / RBP as computing gradients at the steady state of a convergent recurrent network via implicit differentiation — and recognize this as a precursor to modern adjoint and equilibrium methods.

- See DEQs as a modern repackaging of the same idea: prediction as a fixed point $z^* = f_\theta(z^*, x)$ solved by a black-box root finder, with constant-memory gradients via implicit differentiation — decoupling forward-pass depth from backward-pass cost.

- Appreciate how implicit/fixed-point layers can represent some functions much more efficiently than bounded-depth feed-forward nets, at the cost of requiring convergence guarantees and iterative solvers at inference time. Be aware of the practical failure modes: convergence is not guaranteed outside the contraction regime, solvers can be sensitive to initialization and hyperparameters, and truncating the forward iteration introduces bias into the implicit gradients.

- *Bridge to the rest of the course:* Gradient descent itself is a fixed-point iteration (converging to a stationary point of the loss) and a higher-order function (it inspects the loss landscape to decide how to update parameters). We take this for granted at training time. The rest of the course asks: what happens when we bring such iterative, self-adjusting processes to inference time?

1.3 Lecture 3 – Adaptive Computation: Halting, Routing, and Dynamic Depth

When should a network stop thinking? Architectures that learn how much computation to spend per input or token — via halting mechanisms, weight-sharing with adaptive depth, and dynamic routing under fixed FLOP budgets.

1.3.1 Core readings

- Graves, "Adaptive Computation Time for Recurrent Neural Networks" (2016)
- Dehghani et al., "Universal Transformers" (2019)
- Elbayad et al., "Depth-Adaptive Transformer" (2020)
- Raposo et al., "Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models" (2024)

1.3.2 Optional readings

- Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer" (2017) *(Adaptive width via routing — the complementary axis to adaptive depth. MoD can be seen as "Mixture-of-Experts applied to layers rather than sub-networks.")*
- Banino et al., "PonderNet: Learning to Ponder" (2021) *(Revisits ACT's halting mechanism with a probabilistic framework and improved training; good companion to the Graves paper.)*
- Figurnov et al., "Spatially Adaptive Computation Time for Residual Networks" (2017) *(Extends ACT to ConvNets with per-spatial-position halting — shows the idea generalizes beyond sequences to any structured input where different regions require different amounts of processing.)*
- Schuster et al., "Confident Adaptive Language Modeling" (CALM, 2022) *(Early-exit inference for LLMs — adaptive depth at deployment time rather than training time. The network must estimate its own confidence to decide whether to keep computing, previewing L4's introspective theme.)*

1.3.3 Key takeaways

- ACT introduced learnable halting for RNNs: a scalar "pondering" signal that accumulates until the network decides to stop. The Universal Transformer transplants this idea into the Transformer by sharing weights across layers — yielding variable depth per token within a single architecture.

- Depth-Adaptive Transformers and Mixture-of-Depths take a different tack: rather than learning *when to stop*, they learn *which tokens to route through which layers*, achieving adaptive computation under a hard FLOP budget. This reframes the problem from halting to routing.
- DEQs (L2) achieve adaptive depth *implicitly* — iterate until convergence to a fixed point. ACT/UT achieve it *explicitly* — learn a halting distribution. Both break the fixed-depth paradigm, but DEQs offer constant-memory guarantees while ACT-style methods offer more direct control over the compute–accuracy tradeoff.
- Adaptive depth vs. adaptive width (MoE) are complementary axes of the same idea: allocate compute where it's needed. Both can be composed.
- Confidence-based early exit (CALM) hints at what comes next in L4: the network must *introspect* — estimate its own uncertainty — to decide whether to keep computing.
- A recurring practical concern: learned halting and routing decisions can be poorly calibrated — models may learn to use a fixed number of steps regardless of input difficulty, or may halt too early on out-of-distribution inputs. Training the halting/routing signal reliably remains an open problem.

1.4 Lecture 4 – Iterative Refinement and Introspective Computation

Networks that look at their own internal state, refine it iteratively, and decide how much further "thinking" to do. Where L3 asked "how much computation?", L4 asks "what should that computation be doing — and who decides?"

1.4.1 Core readings

- Locatello et al., "Object-Centric Learning with Slot Attention" (2020) *(The simplest and cleanest instance of iterative refinement: slot representations compete for input ownership through repeated attention rounds, converging toward a fixed point. A direct bridge from L2's contraction maps to L4's introspective architectures.)*
- Mathur, Pearlmutter, & Plis, "MIND over Body: Adaptive Thinking using Dynamic Computation" (ICLR 2025) *(Introspection as architectural principle: a secondary network examines the prediction network's*

intermediate activations and outputs a per-layer mask deciding which layers need fixed-point iteration and which pass through in a single step — jointly adapting model complexity and iteration depth per input.)

- Mathur, Doan, Pearlmutter, & Plis, "Change of Thought: Adaptive Test-Time Computation" (SELF-Transformer, 2025) *(Takes the L1 observation that attention matrices are produced on the fly by a higher-order function and extends it to a dynamical process: the alignment matrix itself is iteratively refined via FPI until convergence, with zero additional parameters. The network reasons in latent space without decoding to tokens — contrast with chain-of-thought.)*

- Horne et al., "Continuous Thought Machines" (2025) *(Neural dynamical system where ongoing continuous-time activity and synchronization patterns implement "thought," naturally yielding input-dependent compute via a different mechanism than discrete FPI.)*

1.4.2 Optional readings

- Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" (2022) *(Iterative refinement in token space rather than latent space. CoT is the output-level counterpart to the latent-space iteration in MIND/SELF-Transformer/CTM — an important conceptual contrast.)*

- Chen et al., "Inner Thinking Transformer: Leveraging Dynamic Depth Scaling to Foster Adaptive Internal Thinking" (2025)

- Geiping et al., "Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach" (2025) *(Scales up the lecture's core idea — recurrent depth as test-time compute — to LLM-scale language models.)*

- Wang et al., "Hierarchical Reasoning Model (HRM)" (2025)

- Jolicoeur-Martineau, "Less is More: Recursive Reasoning with Tiny Networks (TRM)" (2025)

1.4.3 Key takeaways

- Iterative refinement has a design space along two axes: *what* is being refined (slot representations, hidden states, alignment matrices, full neural dynamics) and *who decides when to stop* (fixed iteration count,

learned introspection network, convergence criterion, continuous-time evolution).

- Slot Attention (2020) is the simplest instance: representations are iteratively recomputed via competitive attention, converging toward a fixed point — a direct bridge from L2's contraction maps to L4's introspective architectures.

- MIND introduces a qualitatively new capability: the network *introspects on its own internal representations* to decide the structure of its own computation graph. The introspection network examines activations *after* the prediction network has begun processing — so its decisions are conditioned on what the model has already computed, not just on the raw input. This jointly adapts both *which* layers iterate and *how long* they iterate, combining themes from L2 (FPI with convergence guarantees) and L3 (adaptive depth/routing).

- SELF-Transformer picks up a thread from L1: standard attention computes the alignment matrix $\mathcal{T}$ in a single pass (input $\rightarrow$ Q,K $\rightarrow$ softmax $\rightarrow \mathcal{T}$). FPSA turns this into a *dynamical process* where $\mathcal{T}$ itself is iteratively refined to a fixed point. Crucially, what iterates is *the operator* (the alignment matrix), not just the output — the network adjusts its own mixing weights at inference time.

- Token-space refinement (Chain-of-Thought) vs. latent-space refinement (MIND/SELF-Transformer/CTM) represents a fundamental design choice: reasoning in the output where it is interpretable but expensive in tokens, or in the hidden state where it is compact but opaque. SELF-Transformer's latent reasoning is especially relevant for resource-constrained on-device models where token-level autoregressive reasoning is prohibitively expensive.

- CTM offers a complementary perspective: rather than discrete fixed-point iterations, computation unfolds as continuous-time neural dynamics where synchronization patterns implement "thought." This connects back to L1's view of RNNs as dynamical systems, but with richer temporal structure.

1.5 Lecture 5 – Memory, State Tracking, and Long-Horizon Reasoning

From neural networks with scratch pads (NTM) to models that learn to memorize at test time (Titans), and what the RNN-vs-Transformer debate reveals about inductive biases for state tracking.

1.5.1 Core readings

- Graves, Wayne, & Danihelka, "Neural Turing Machines" (2014)
- Behrouz et al., "Titans: Learning to Memorize at Test Time" (2025)
- Hu et al., "Every Token Counts: Generalizing 16M Ultra-Long Context in LLMs" (2025)
- Ebrahimi et al., "On the 'Induction Bias' in Sequence Models" (2026)

1.5.2 Optional readings

- Weston, Chopra, & Bordes, "Memory Networks" (2014)
- Graves et al., "Hybrid computing using a neural network with dynamic external memory" (2016)
- Ramsauer et al., "Hopfield Networks is All You Need" (2020) *(Transformer attention as a modern Hopfield update rule — bridges L2's fixed-point associative memories with L5's attention-as-memory.)*
- Schöne et al., "Implicit Language Models are RNNs" (2025) *(Bridges L2 and L5: implicit/fixed-point layers are recurrent networks, connecting the DEQ theme to state tracking.)*
- Behrouz et al., "Memory Caching: RNNs with Growing Memory" (2026) *(Interpolates between fixed-memory RNNs and growing-memory Transformers via cached hidden-state checkpoints.)*
- Azarafrooz, "Differentiable Neural Computers with Memory Demon" (2022)

1.5.3 Key takeaways

- NTM introduced differentiable external memory — a scratch pad accessed via attention-based read/write heads, breaking the fixed-size bottleneck of RNN hidden states.
- Titans compresses history into fixed-size parameters via gradient-based updates *at test time*, with learned forgetting to manage capacity. This is the L2 observation made literal: gradient descent — the pre-defined second-order process we have always relied on at train time — now runs at inference, adjusting the model on the fly. L6 generalizes this further.
- This gives a design spectrum for neural memory: growing external storage (NTM/DNC), cached hidden-state checkpoints (Memory Caching), or compression into fixed parameters with forgetting (Titans). Each makes a different capacity/cost tradeoff.
- The induction bias paper is an empirical payoff of the extensional/intensional distinction from L2: Transformers appear to learn a *heuristic per input length* (a function), while RNNs learn a *computation* (a process) — now demonstrated empirically on state-tracking tasks.
- Hopfield networks (1982) are the earliest "memory as fixed point"; Ramsauer et al. show attention *is* a modern Hopfield update — connecting L2's fixed-point theme to L5's memory theme.

1.6 Lecture 6 – Test-Time Learning, Self-Modification, and Nested Learners

What happens when we stop hand-designing update rules and let models learn their own — turning the optimization process itself into a learnable, differentiable component?

1.6.1 Core readings

- Finn, Abbeel & Levine, "Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks" (2017)
- Sun et al., "Learning to (Learn at Test Time): RNNs with Expressive Hidden States" (2024)
- Behrouz et al., "Nested Learning: The Illusion of Deep Learning Architectures" (2025)

1.6.2 Optional readings

- Schmidhuber, "Steps Towards Self-Referential Neural Learning" (1992) — a network that learns to modify its own weights via self-referential weight changes; earliest concrete mechanism for what this lecture generalizes
- Schmidhuber, "A Neural Network that Embeds its Own Meta-Levels" (1993) — extends the self-referential idea: a single network implements both base learning and meta-learning, collapsing the hierarchy; direct ancestor of nested/learned update rules
- Rajeswaran et al., "Meta-Learning with Implicit Gradients (iMAML)" (2019) — replaces MAML's unrolled differentiation with implicit differentiation at the inner-loop fixed point; direct callback to L2's IFT machinery
- Hübotter et al., "Efficiently Learning at Test-Time: Active Fine-Tuning of LLMs (SIFT)" (2024) — selective, data-efficient test-time fine-tuning for LLMs; practical scaling of TTT ideas
- ArXivIQ blog explainer on Nested Learning and HOPE — accessible walk-through of the nested optimization framing
- Adaptive Resonance Theory (Wikipedia) — Grossberg's classical framework for self-stabilizing, self-organizing learning; context for the "nested learners at different time scales" idea
- Grossberg, "Adaptive Resonance Theory: How a brain learns to consciously attend, learn, and recognize a changing world" (2013) — full ART treatment; relevant to vigilance-gated self-modification
- Grossberg, "Adaptive resonance theory" (Scholarpedia overview) — concise ART reference

1.6.3 Key takeaways

- MAML makes the inner optimization loop explicit and differentiable: a meta-learner produces initializations from which a few gradient steps yield a task-specific learner. This is a higher-order function (a learner) over learners — extending the motif from attention (L1) through introspection networks (L4/MIND) to its most general form.

- TTT layers bring the meta-learning idea inside the forward pass: each layer runs a self-supervised gradient update on its own hidden state for every test sequence. This makes literal what L2 planted and L5's Titans demonstrated — gradient descent, the second-order fixed-point iteration we take for granted at train time, now runs at inference — but here the update rule's loss function is itself learned.

- Nested Learning reframes the entire design space: models, memory modules, and optimizers are all associative memories / learners operating at different time scales (fast weights, slow weights, persistent memory). HOPE instantiates this as a self-modifying sequence model with continuum memory. This is one proposal, not settled theory — but it offers a clean lens for retrospectively organizing the course.

- iMAML closes a circle with L2: MAML's inner loop converges to a fixed point; iMAML differentiates through that fixed point implicitly (IFT) rather than unrolling. This is exactly the Almeida–Pineda move — implicit differentiation at equilibrium — now applied to the meta-learning inner loop. MAML unrolls; iMAML recognizes the fixed point and uses IFT, just as DEQs apply the same Almeida–Pineda insight to arbitrary depth-tied architectures with a black-box root finder.

- The full arc, viewed through the extensional/intensional lens from L2: L1 models approximate fixed functions (extensional); L2–L5 progressively internalize iterative processes (fixed-point solving, adaptive halting, introspection, test-time memory); L6 reaches the deepest intensional level — models that learn *processes that learn*, where the update rule itself is a learned, differentiable object.

- Design spectrum for test-time adaptation: pre-defined update rule on learned representations (Titans) → learned initialization with pre-defined inner loop (MAML) → learned update rule (TTT) → nested hierarchy of learned update rules at multiple time scales (Nested Learning/HOPE). Each step trades simplicity/guarantees for expressiveness.